

GSoC21 W9: LFortran Bug Hunting Bonanza

Rohit Goswami · 2021-08-06 · gsoc21 · fortran · ramblings

Continuing compile time intrinsic functions, ASR passes, and testing LAPACK

Background

Serialized update for the [2021 Google Summer of Code](#) under the [fortran-lang organization](#), mentored by [Ondrej Certik](#).

Logistics

- Daily checkins over Zulip
 - Code reviews on Gitlab
- Met with Ondrej and Thirumalai on Thursday
 - Discussed comments and how to attach them
 - Decided that it would make sense to possibly store the comments in the ASR to make sure that the comments are attached to the right nodes
- We discussed a set of outputs for the next week and wrap up policies
 - It was fun to see some of Thirumalai's work in person!

Overview

This has been a bit of a slow slog. With the `fixed form parser` completed, LAPACK is no longer a distant dream, so this week involved tracing through LAPACK and narrowing down issues for it. There are a few open issues which have arcane sources; but these are meant to be handled with prejudice and not merely complained about.

New Merge Requests

I have decided to elide draft MRs for now.

[tiny: Body visitor implementation \(1112\)](#)

The population of `value` for `tiny` calls beyond the symbol table i.e. in the body

Freshly Assigned Issues

[-show-asr Scientific notation \(480\)](#)

A visual glitch in the prettied output; not really under my ambit but it might be fun

[Compiling LAPACK \(481\)](#)

This is the first fixed form program to be tested, there are some bugs to be ironed out but it should be very much within what we can do by the end of the program. This issue is mostly handled by Ondrej, but since it is application based I assigned myself as well

Namespace proposal for compile time intrinsics (482)

This is part of a refactoring design plan

Semantics: Implement REAL(A [, KIND]) (483)

Conversions! The bedrock of many a numerical program, this is one of the most relevant casts

Semantics: Implement INT(A [, KIND]) (484)

All conversions are similar, and sort of pointless in the SymbolTable, but still necessary and fun

Formatting: Array pointout (485)

Another minor visual glitch

Misc Logs

LAPACK Setup

We start with a setup similar to that of `minidftatom` and `dftatom`.

```
git clone git@github.com:Reference-LAPACK/lapack.git
cd lapack
git checkout v3.10.0
```

Now we can build it with `gfortran` as a test.

```
mkdir build && cd build
ccmake .. # Make sure to turn on testing
FC=$(which gfortran) cmake .. -DCMAKE_INSTALL_LIBDIR=$(pwd)/tmp -G "Ninja"
cmake --build .
ctest
```

This is a rather long process, with `4097` files to compile, `6814` if `BLAS` is required; the dependency tree is bound to be rather esoteric looking; so we won't bother.

Rather than using `cmake` for the `lfortran` test we will use the `makefile` setup.

```
cp make.inc.example make.inc # Modify to set paths
```

Well the very first issue is that we should remove the `doxygen` style comments which are marked by `*>`.

Actually we should go ahead and remove all lines starting with `*`. The [regex for this](#) is `\*.*$` which simply captures any line starting with `*`. I use `sd` (a [grep alternative](#)) but any standard tool will work, including `grep -vE`.

```
# Stadarnd
cat blah.f | grep -vE '^\*'
# Easier
# cargo install sd
for file in ./*.f; do
    sd '\*.*$' '' $file
done
```

Real Body Visitors

I spent far too long staring at code which actually errored out for a different reason. The MR logic is solid, but `Var` doesn't have a value assigned to it yet, even when it is decorated with `parameter` which was totally a silly mistake on my end. `parameter`, as Ondrej pointed out, effectively acts as though the variable

is declared to be a constant. So there is no conceptual problem assigning an `expr_t* value` to such a statement.

Not sure why the standard allows this, but it felt incredibly pointless to implement `real` for the symbol-table:

```
program main
  implicit none
  integer, parameter :: x=3
  real, parameter :: y=real(x)
  print*, y
end program main
```

Which `gfortran` accepts without a complaint. Not really much of a pain since there is a proposed refactor which is one of the primary goals of the next week after the bug fixes are in but still odd.

Conclusions

Next week is critical. The LLVM backend must gracefully accept `value` when present, and there are some additional tasks which simply must be completed. All the QoL MRs will be in next week as well. That said, I expect at this point to dedicate more time to the project, which is not a bad thing, but a bit of a sobering realization. Last week I wasn't working over the weekend, but I have several catchup tasks to finish this time around; for draft MRs and issues.